



Non Bit-blasting Bit-vector Solving

Pedro Saccomani, Vinícius dos Santos Daher Magri

Universidade Federal de Minas Gerais

October 9, 2026

Motivation

Bit-vectors capture the semantics of machine integers: k-bit integers can be modeled as bit-vectors of width k:

$$a = a_{k-1}a_{k-2}\cdots a_1 a_0 \quad \longleftrightarrow \quad a^{\text{int}} = \sum_{i=0}^{k-1} a_i \cdot 2^i$$

Arithmetic operations over bit-vectors then correspond to arithmetic modulo 2^k over the integers.

Introduction: Deciding bit-vector constraints

How does an SMT solver decide a bit-vector formula?



- φ: input formula in the bit-vector theory.
- φ^{bv}: the conjunction of bit-vector *literals* whose satisfiability the theory solver must decide.
- φ^{prop}: an equisatisfiable propositional (CNF) formula.

Introduction: Bit-blasting

The step $\varphi^{\mathsf{bv}} \to \varphi^{\mathsf{prop}}$ is called bit-blasting:

- each term of width k becomes k Boolean variables;

Introduction: Bit-blasting

The step $\varphi^{\mathsf{bv}} \to \varphi^{\mathsf{prop}}$ is called bit-blasting:

- each term of width k becomes k Boolean variables;
- each predicate is encoded as an equisatisfiable set of clauses;

Introduction: Bit-blasting

The step $\varphi^{\mathsf{bv}} \to \varphi^{\mathsf{prop}}$ is called bit-blasting:

- each term of width k becomes k Boolean variables;
- each predicate is encoded as an equisatisfiable set of clauses;
- each operator is encoded by a Boolean circuit.

Introduction: Bit-blasting

The step $\varphi^{\text{bv}} \rightarrow \varphi^{\text{prop}}$ is called bit-blasting:

- each term of width k becomes k Boolean variables;
- each predicate is encoded as an equisatisfiable set of clauses;
- each operator is encoded by a Boolean circuit.

SMT-LIB's fixed-size bit-vector theory has **many** operators to encode:

```
Bitwise:  bvand, bvor, bvxor, bvnot, bvnand, bvnor, bvxnor
Arithmetic:  bvadd, bvsub, bvneg, bvmul, bvudiv, bvurem, bvsdiv, bvsrem,
bvsmod
Shifts:  bvshl, bvlshr, bvashr
Structural:  concat, extract, zero_extend, sign_extend, repeat,
rotate_left, rotate_right
Relational:  bvult, bvule, bvugt, bvuge, bvslt, bvsle, bvsgt, bvsge, =,
distinct
```

Example: bit-blasting a 2-bit formula

Input (x, y of width 2):

$$\varphi^{\mathsf{bv}} \;=\; (\texttt{bvand}\ x\ y) \approx \texttt{\#b01}$$

Example: bit-blasting a 2-bit formula

Input (x, y of width 2):

$$\varphi^{\mathsf{bv}} \;=\; (\texttt{bvand}\ x\ y) \approx \texttt{\#b01}$$

1. Bits of each term

$x \mapsto x_1 x_0 \qquad y \mapsto y_1 y_0$

$(\texttt{bvand}\ x\ y) \mapsto r_1 r_0$

Example: bit-blasting a 2-bit formula

Input (x, y of width 2):

$$\varphi^{\mathsf{bv}} \;=\; (\texttt{bvand}\ x\ y) \approx \texttt{\#b01}$$

1. **Bits of each term**

$$x \mapsto x_1 x_0 \qquad y \mapsto y_1 y_0$$

$$(\texttt{bvand}\ x\ y) \mapsto r_1 r_0$$

2. **Predicate** ($\approx$ #b01)

$$r_0 \;\wedge\; \neg r_1$$

Example: bit-blasting a 2-bit formula

Input (x, y of width 2):

$$\varphi^{\mathsf{bv}} \;=\; (\texttt{bvand}\ x\ y) \approx \#\texttt{b01}$$

1. **Bits of each term**

$$x \mapsto x_1 x_0 \qquad y \mapsto y_1 y_0$$

$$(\texttt{bvand}\ x\ y) \mapsto r_1 r_0$$

2. **Predicate** ($\approx \#\texttt{b01}$)

$$r_0 \;\wedge\; \neg r_1$$

3. **Circuit for** $\texttt{bvand}$ $(r_i \Leftrightarrow x_i \wedge y_i)$

$$(\neg r_i \vee x_i) \quad (\neg r_i \vee y_i) \quad (r_i \vee \neg x_i \vee \neg y_i)$$

$$\text{for } i \in \{0, 1\}$$

Example: bit-blasting a 2-bit formula

Input (x, y of width 2):

$$\varphi^{\mathsf{bv}} \;=\; (\texttt{bvand}\ x\ y) \approx \#\texttt{b01}$$

1. Bits of each term

$$x \mapsto x_1 x_0 \qquad y \mapsto y_1 y_0$$

$$(\texttt{bvand}\ x\ y) \mapsto r_1 r_0$$

2. Predicate ($\approx \#\texttt{b01}$)

$$r_0 \;\wedge\; \neg r_1$$

3. Circuit for $\texttt{bvand}$ $(r_i \Leftrightarrow x_i \wedge y_i)$

$$(\neg r_i \vee x_i) \quad (\neg r_i \vee y_i) \quad (r_i \vee \neg x_i \vee \neg y_i)$$

$$\text{for } i \in \{0, 1\}$$

Result: φ^{prop} has 6 variables and 8 clauses, and is satisfied by $x = \#\texttt{b01},\ y = \#\texttt{b11}$.

Introduction: Arithmetic as a bottleneck



$\mathcal{O}(k^2)$ clauses

- The multiplication circuit, and hence its CNF encoding, is **quadratic in the width** k.

Introduction: Arithmetic as a bottleneck



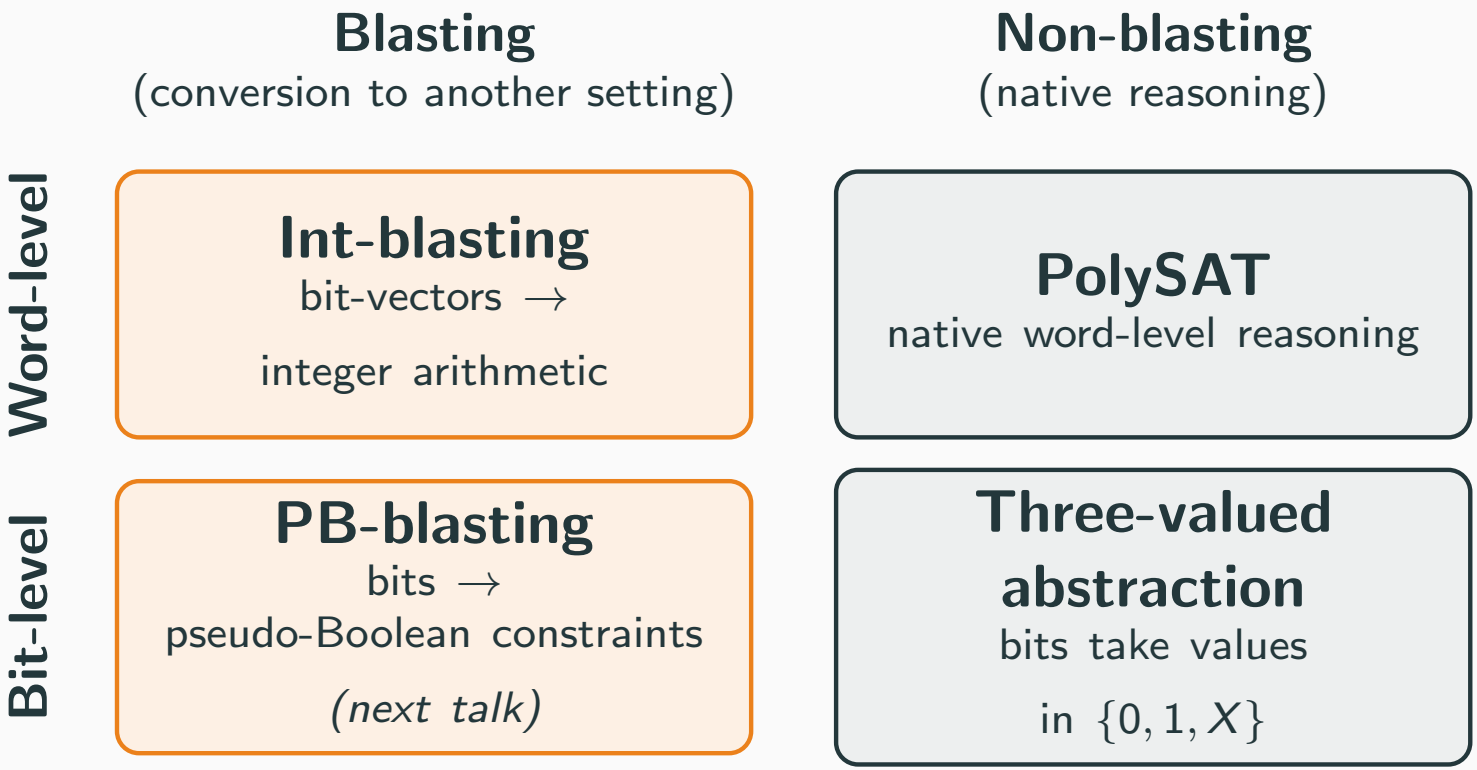
$\mathcal{O}(k^2)$ clauses

- The multiplication circuit, and hence its CNF encoding, is **quadratic in the width** k.
- Worse: **resolution refutations of multiplication identities can be exponential**.

A. Biere, *Weaknesses of CDCL Solvers*.
A. Niemetz, *Challenges in State-of-the-Art Bit-Precise Reasoning*.

Outline

In this seminar, we will go through four different techniques, working at different levels of abstraction:



Three-valued abstraction - Setting

Based on abstract interpretation: bit-vectors are abstracted into a three-valued domain.

Three-valued abstraction - Setting

Based on abstract interpretation: bit-vectors are abstracted into a three-valued domain.

- **Bits** are abstracted as values from $\mathbb{X} = \{\underbrace{\{0\}}_{0}, \underbrace{\{1\}}_{1}, \underbrace{\{0,1\}}_{X}\}$.

Three-valued abstraction - Setting

Based on abstract interpretation: bit-vectors are abstracted into a three-valued domain.

- **Bits** are abstracted as values from $\mathbb{X} = \{\underbrace{\{0\}}_{0}, \underbrace{\{1\}}_{1}, \underbrace{\{0,1\}}_{X}\}$.
- **Bit-vectors** are abstracted as sequences of values in $\mathbb{X}$.

Three-valued abstraction - Setting

Based on abstract interpretation: bit-vectors are abstracted into a three-valued domain.

- **Bits** are abstracted as values from $\mathbb{X} = \{\underbrace{\{0\}}_{0}, \underbrace{\{1\}}_{1}, \underbrace{\{0,1\}}_{X}\}$.
- **Bit-vectors** are abstracted as sequences of values in $\mathbb{X}$.

Key idea: lift operators over plain bit-vectors to this three-valued domain.

Three-valued abstraction - Lifting

Some operators have simple liftings: `bvand` is bitwise conjunction in three-valued logic .

$\wedge$	0	1	X
0	0	0	0
1	0	1	X
X	0	X	X

Three-valued abstraction - Lifting

Some operators have simple liftings: `bvand` is bitwise conjunction in three-valued logic .

$\wedge$	0	1	X
0	0	0	0
1	0	1	X
X	0	X	X

Arithmetic operators, on the other hand, have non-trivial liftings:

Three-valued abstraction - Lifting

Some operators have simple liftings: `bvand` is bitwise conjunction in three-valued logic .

$\wedge$	0	1	X
0	0	0	0
1	0	1	X
X	0	X	X

Arithmetic operators, on the other hand, have non-trivial liftings:

[**Onderka and Ratschan**, **2022**] introduced a **new** technique to compute them, called *modular extreme-finding*.

Three-valued abstraction - Examples

Evaluating bit-vector operations in the three-valued domain [**Onderka et al.**, **2026**]:

Op.	Op. 1	Op. 2	Result
bvnot	"0X1X"	-	"1X0X"
bvand	"0X1X"	"1011"	"001X"
bvor	"0X1X"	"1011"	"1X11"
bvxor	"0X1X"	"1011"	"1X0X"
bvadd	"00X0"	"0011"	"0XX1"
bvsub	"00X0"	"0011"	"11X1"
bvmul	"001X"	"0010"	"01X0"
bvudiv	"110X"	"1010"	"0001"
bvurem	"110X"	"1010"	"001X"
bvsdiv	"110X"	"1010"	"000X"
bvsrem	"110X"	"1010"	"XXXX"
bvsmod	"110X"	"1010"	"XXXX"

Op.	Op. 1	Op. 2	Result
=	"0000"	"00X0"	"X"
bvult	"00X0"	"010X"	"1"
zero_extend	2	"X1"	"00X1"
sign_extend	2	"X1"	"XXX1"
bvshl	"0010"	"000X"	"0XX0"
bvlshr	"0010"	"000X"	"00XX"
bvashr	"1010"	"X000"	"1X1X"

Three-valued abstraction - Search

In this new domain, a DPLL-based search algorithm is then applied, with additional machinery for backjumping and learning inspired by CDCL.

Three-valued abstraction - Search

In this new domain, a DPLL-based search algorithm is then applied, with additional machinery for backjumping and learning inspired by CDCL.

Example [**Onderka et al.**, **2026**]: is there an x such that

$$\underbrace{x}_{\text{“XXX”}} \neq 000_2 \ \wedge \ \Big(\underbrace{(x \wedge 100_2)}_{\text{“X00”}} = 000_2 \Big) \ \wedge \ \underbrace{x + x}_{\text{“XXX”}} = 000_2 \ ?$$

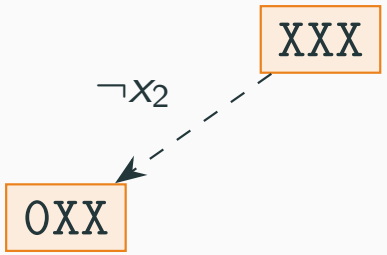
XXX

Three-valued abstraction - Search

In this new domain, a DPLL-based search algorithm is then applied, with additional machinery for backjumping and learning inspired by CDCL.

Example [Onderka et al., 2026]: is there an x such that

$$\underbrace{x}_{\text{“XXX”}} \neq 000_2 \ \wedge \ \Big(\underbrace{(x \wedge 100_2)}_{\text{“X00”}} = 000_2 \Big) \ \wedge \ \underbrace{x + x}_{\text{“XXX”}} = 000_2 \ ?$$

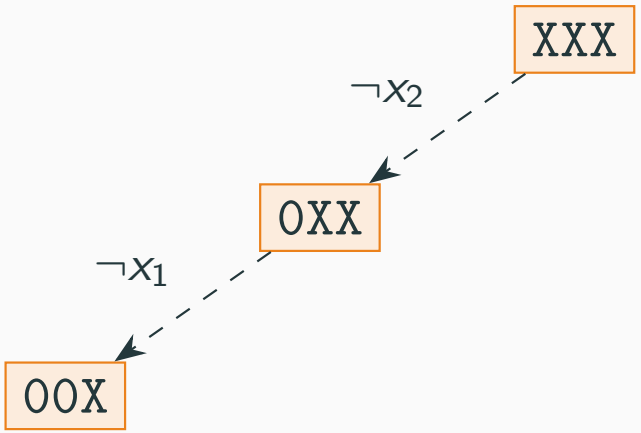


Three-valued abstraction - Search

In this new domain, a DPLL-based search algorithm is then applied, with additional machinery for backjumping and learning inspired by CDCL.

Example [Onderka et al., 2026]: is there an x such that

$$\underbrace{x}_{\text{“XXX”}} \neq 000_2 \ \wedge \ \Big(\underbrace{(x \wedge 100_2)}_{\text{“X00”}} = 000_2 \Big) \ \wedge \ \underbrace{x + x}_{\text{“XXX”}} = 000_2 \ ?$$

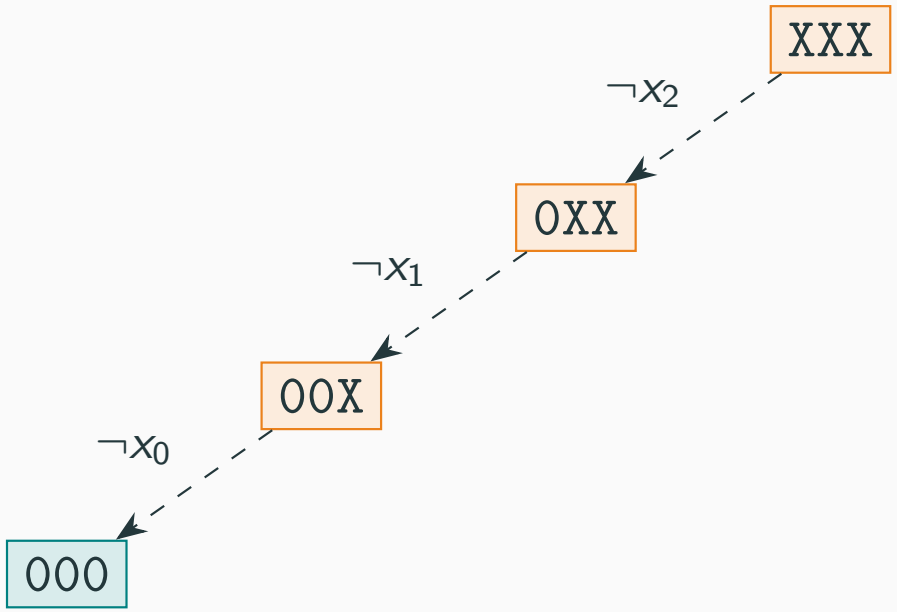


Three-valued abstraction - Search

In this new domain, a DPLL-based search algorithm is then applied, with additional machinery for backjumping and learning inspired by CDCL.

Example [Onderka et al., 2026]: is there an x such that

$$\underbrace{x}_{\text{“XXX”}} \neq 000_2 \ \wedge \ \Big(\underbrace{(x \wedge 100_2)}_{\text{“X00”}} = 000_2 \Big) \ \wedge \ \underbrace{x + x}_{\text{“XXX”}} = 000_2 \ ?$$

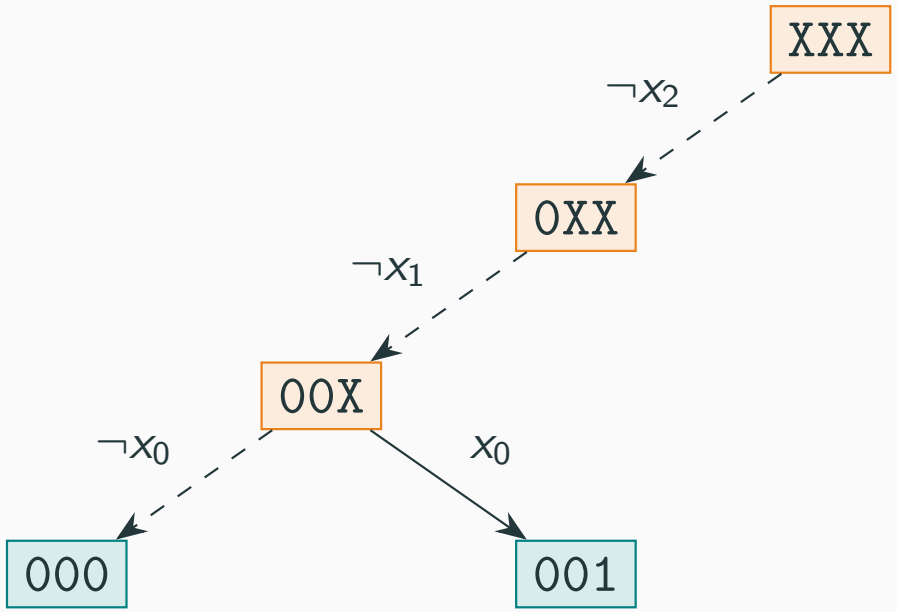


Three-valued abstraction - Search

In this new domain, a DPLL-based search algorithm is then applied, with additional machinery for backjumping and learning inspired by CDCL.

Example [Onderka et al., 2026]: is there an x such that

$$\underbrace{x}_{\text{“XXX”}} \neq 000_2 \ \wedge \ \Big(\underbrace{(x \wedge 100_2)}_{\text{“X00”}} = 000_2 \Big) \ \wedge \ \underbrace{x + x}_{\text{“XXX”}} = 000_2 \ ?$$

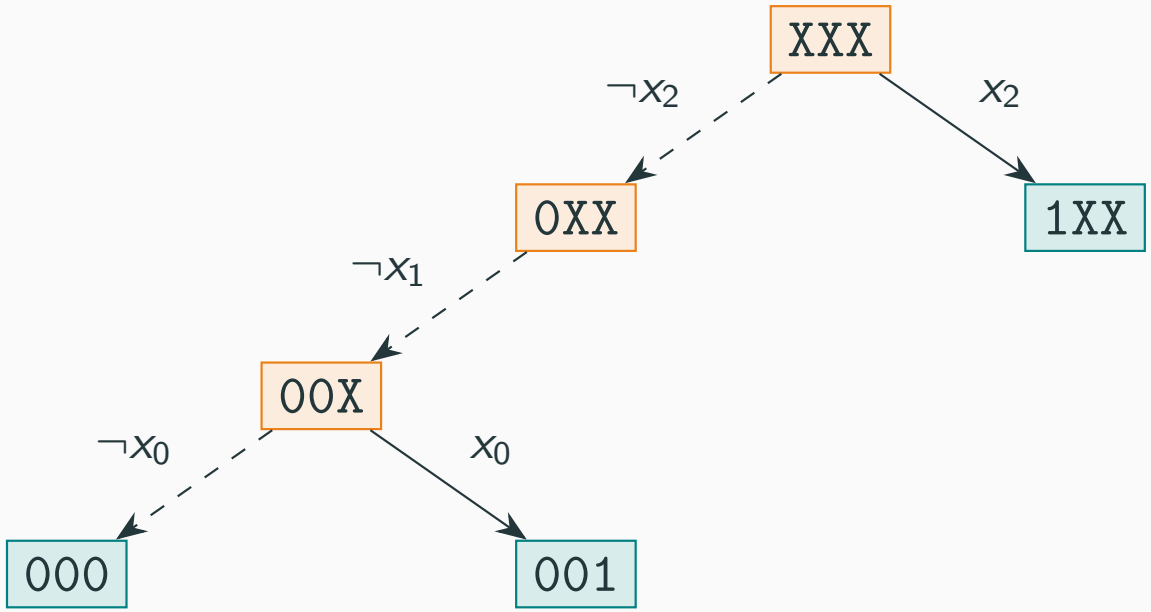


Three-valued abstraction - Search

In this new domain, a DPLL-based search algorithm is then applied, with additional machinery for backjumping and learning inspired by CDCL.

Example [Onderka et al., 2026]: is there an x such that

$$\underbrace{x}_{\text{“XXX”}} \neq 000_2 \ \wedge \ \Big(\underbrace{(x \wedge 100_2)}_{\text{“X00”}} = 000_2 \Big) \ \wedge \ \underbrace{x + x}_{\text{“XXX”}} = 000_2 \ ?$$

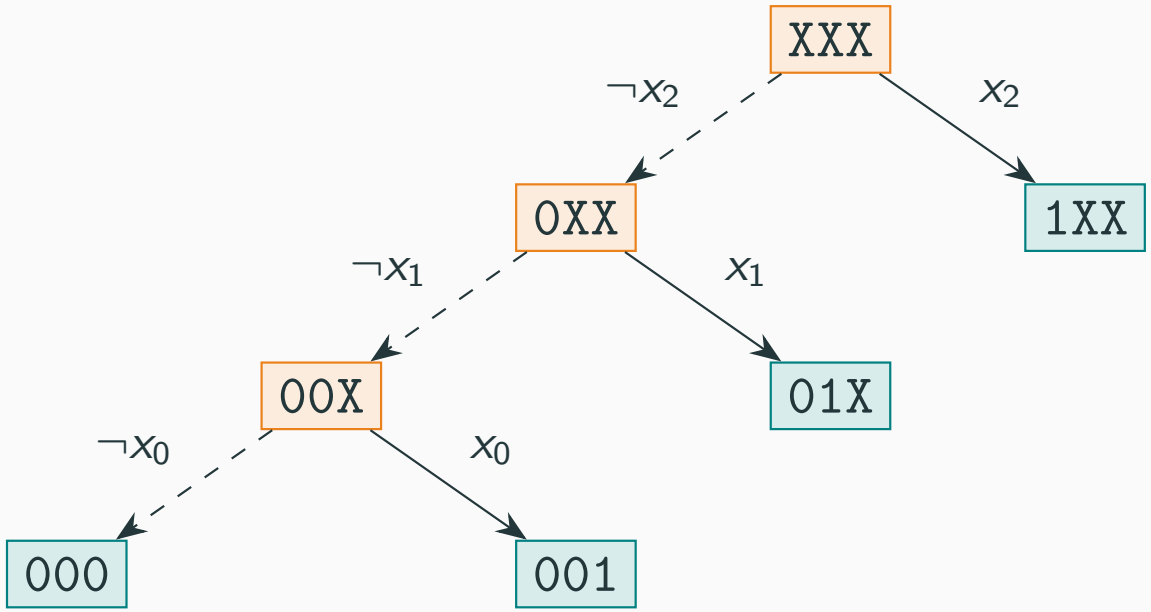


Three-valued abstraction - Search

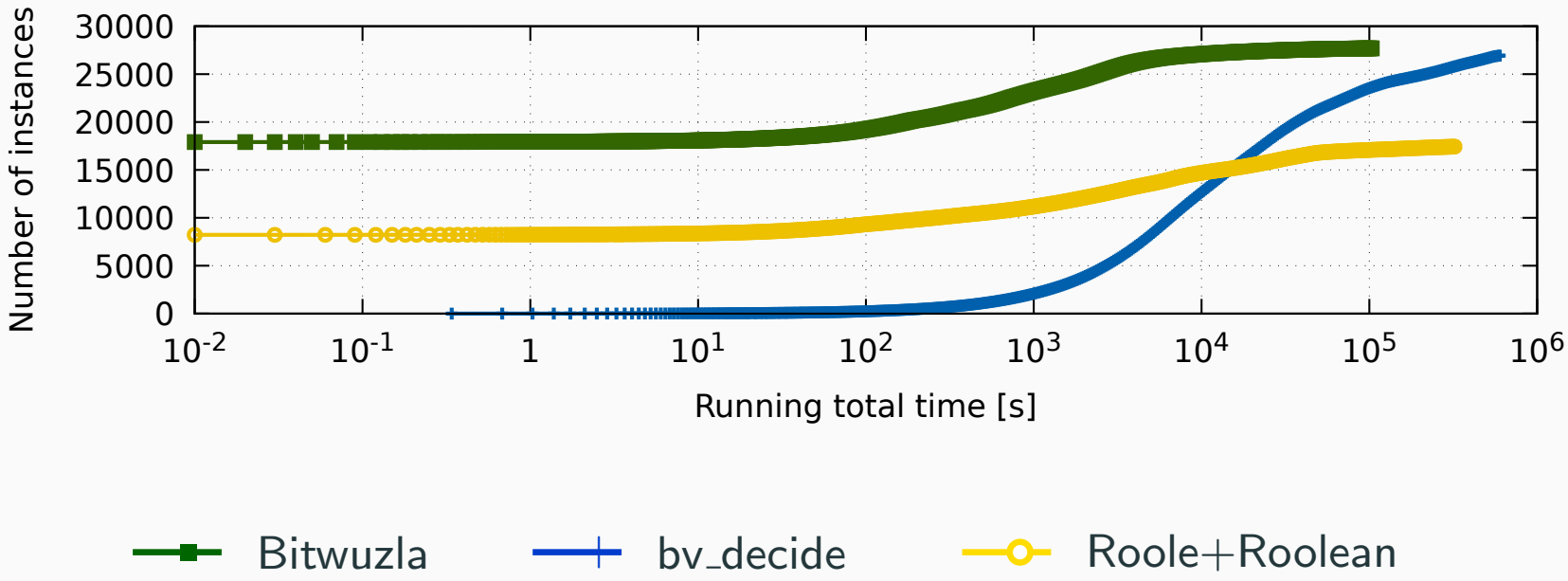
In this new domain, a DPLL-based search algorithm is then applied, with additional machinery for backjumping and learning inspired by CDCL.

Example [Onderka et al., 2026]: is there an x such that

$$\underbrace{x}_{\text{“XXX”}} \neq 000_2 \ \wedge \ \Big(\underbrace{(x \wedge 100_2)}_{\text{“X00”}} = 000_2 \Big) \ \wedge \ \underbrace{x + x}_{\text{“XXX”}} = 000_2 \ ?$$



Three-valued abstraction - Results



Cumulative CPU time of solved **unsat** SMT-LIB 2024 `QF_BV` benchmarks (1200 s, 8 GB per task) [**Onderka et al.**, **2026**].

Three-valued abstraction - Limitations

- **No preprocessing**: Roole applies no preprocessing (rewriting, simplification) to the input.

Three-valued abstraction - Limitations

- **No preprocessing**: Roole applies no preprocessing (rewriting, simplification) to the input.
- **Coarse operators**: `bvsmod` with any "`X`" operand, and `bvurem` with an "`X`" in the quotient, return only "`X`" bits.

PolySAT - Setting

Interpret bit-vectors as algebraic objects: a bit-vector of width k is an element of $\mathbb{Z}/2^k\mathbb{Z}$[1].

[1] Working in this setting is not convenient algorithmically...

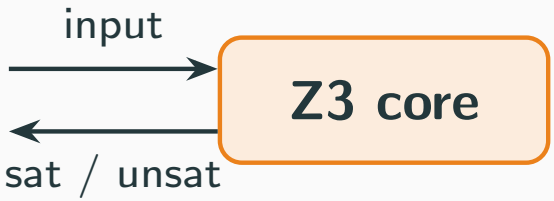
PolySAT - Setting

Interpret bit-vectors as algebraic objects: a bit-vector of width k is an element of $\mathbb{Z}/2^k\mathbb{Z}$[1].

Idea (based on an MCSAT approach): keep track of *forbidden intervals* for bit-vector variables, and assign values outside them.

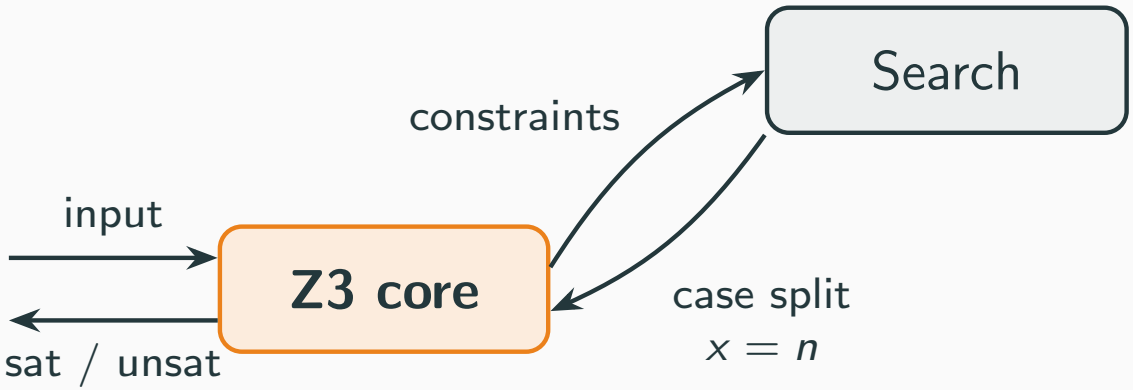
[1] Working in this setting is not convenient algorithmically...

PolySAT - Architecture



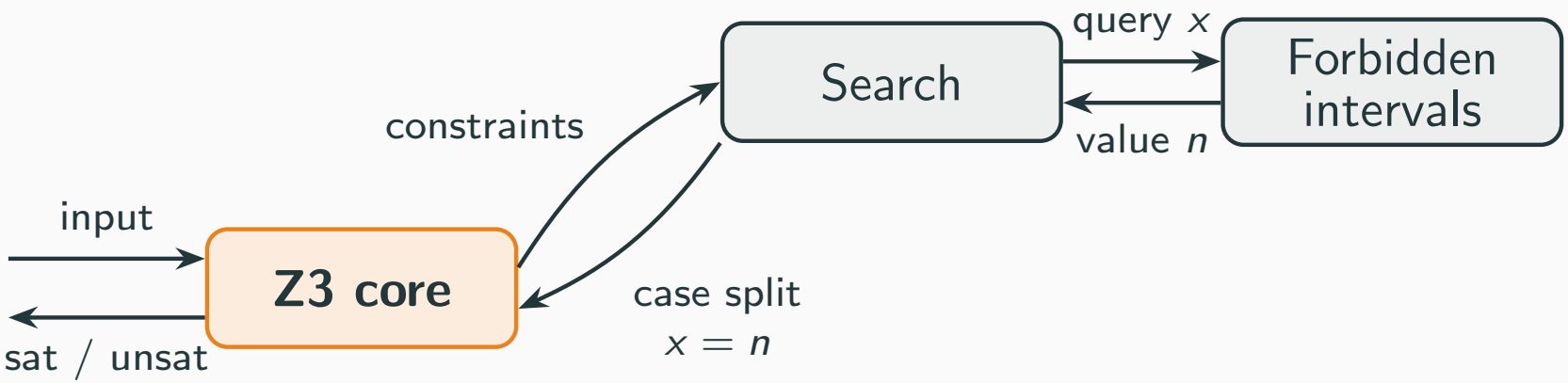
Adapted from [Rath et al., 2024].

PolySAT - Architecture



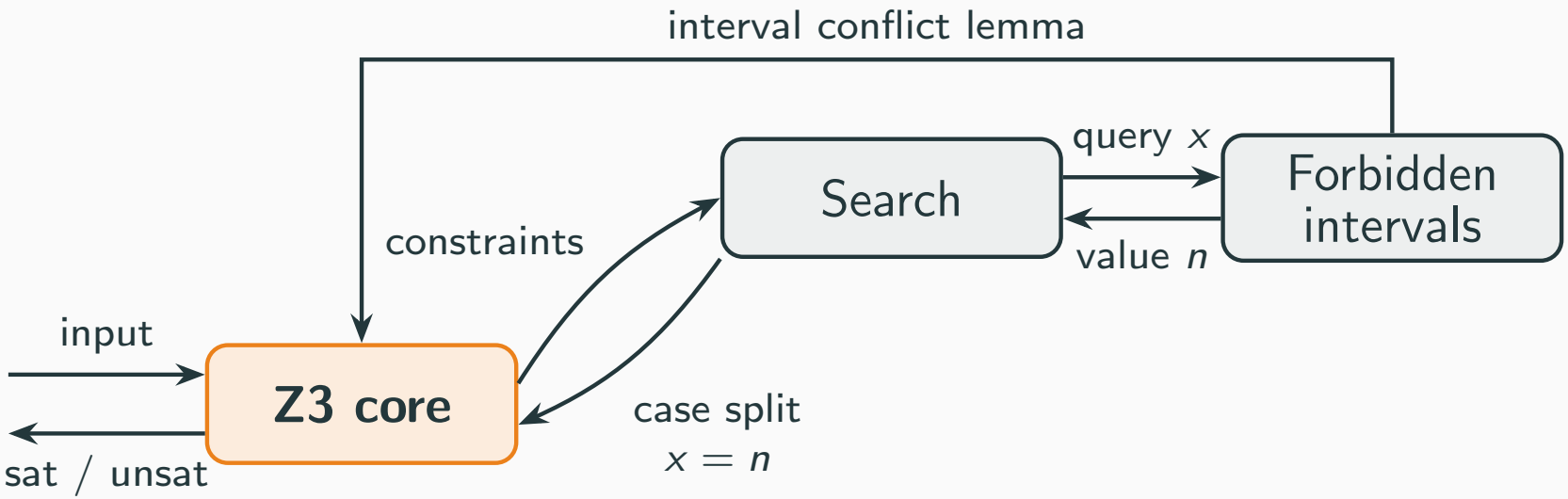
Adapted from [Rath et al., 2024].

PolySAT - Architecture



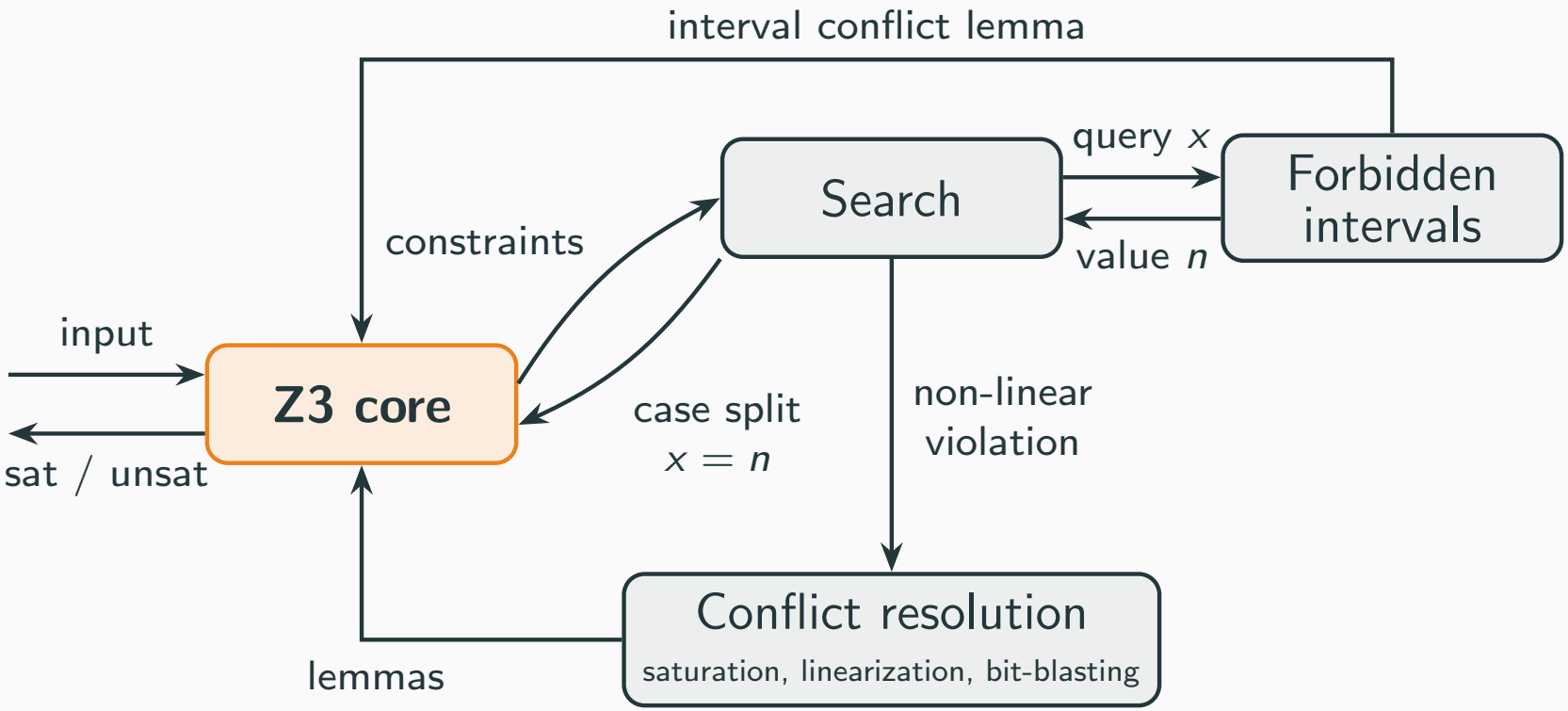
Adapted from [Rath et al., 2024].

PolySAT - Architecture



Adapted from [Rath et al., 2024].

PolySAT - Architecture



Adapted from [Rath et al., 2024].

PolySAT - Forbidden intervals

Each variable x keeps a set of **forbidden intervals** over $\mathbb{Z}/2^k\mathbb{Z}$.

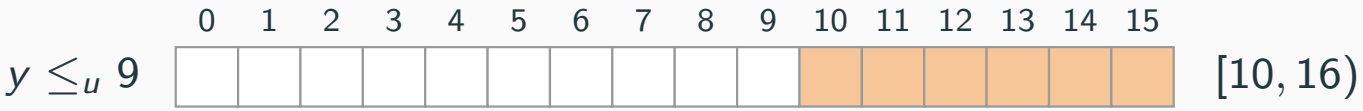
Example ($k = 4$): $y \leq_u 9$, $x + y \geq_u 10$, with $x = 2$ assigned during search.

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

PolySAT - Forbidden intervals

Each variable x keeps a set of **forbidden intervals** over $\mathbb{Z}/2^k\mathbb{Z}$.

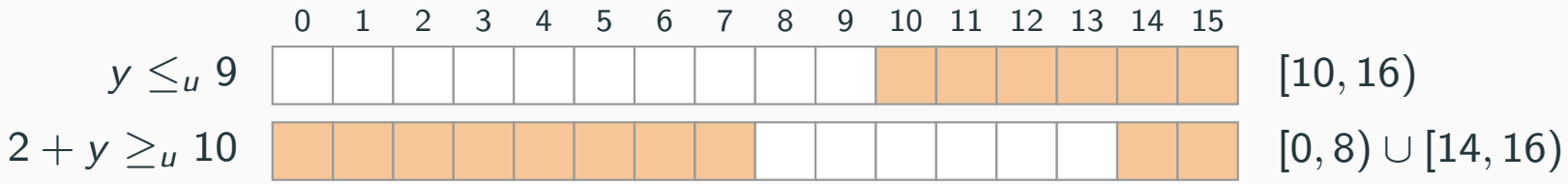
Example ($k = 4$): $y \leq_u 9$, $x + y \geq_u 10$, with $x = 2$ assigned during search.



PolySAT - Forbidden intervals

Each variable x keeps a set of **forbidden intervals** over $\mathbb{Z}/2^k\mathbb{Z}$.

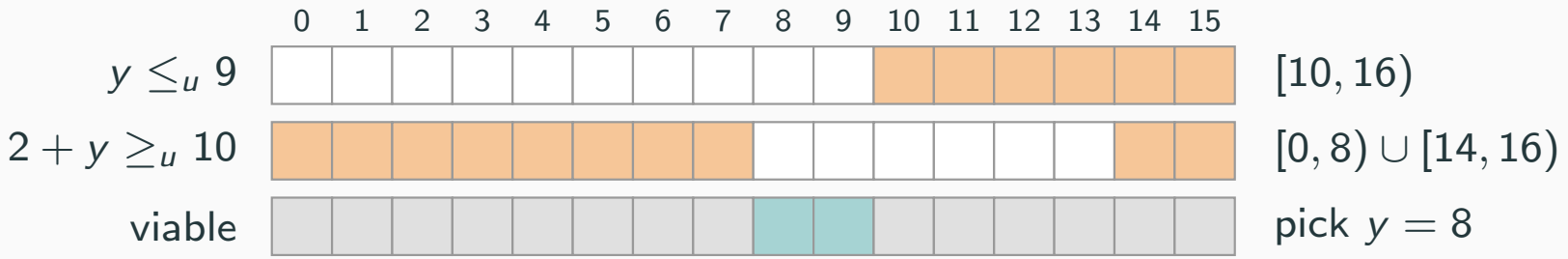
Example ($k = 4$): $y \leq_u 9$, $x + y \geq_u 10$, with $x = 2$ assigned during search.



PolySAT - Forbidden intervals

Each variable x keeps a set of **forbidden intervals** over $\mathbb{Z}/2^k\mathbb{Z}$.

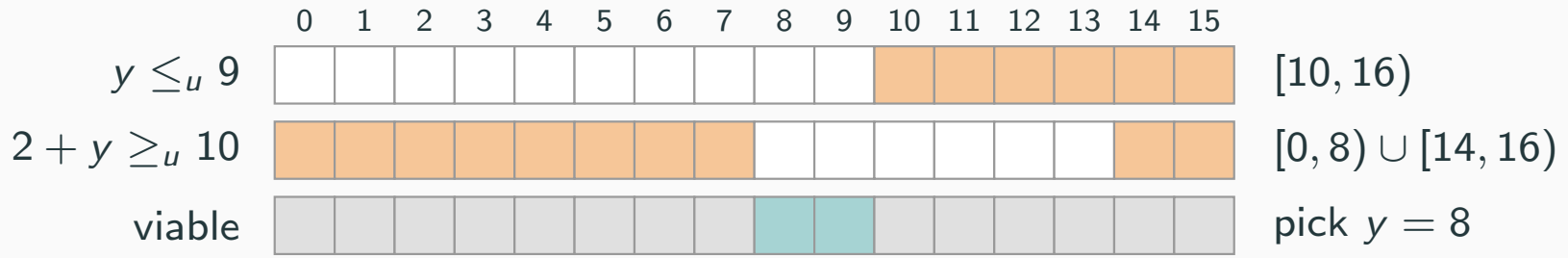
Example ($k = 4$): $y \leq_u 9$, $x + y \geq_u 10$, with $x = 2$ assigned during search.



PolySAT - Forbidden intervals

Each variable x keeps a set of **forbidden intervals** over $\mathbb{Z}/2^k\mathbb{Z}$.

Example ($k = 4$): $y \leq_u 9$, $x + y \geq_u 10$, with $x = 2$ assigned during search.



If the intervals **cover** every value, a lemma built from the covering chain goes back to the core (**interval conflict**).

PolySAT - Conflict resolution

With **all** variables assigned and no interval conflict, constraints are **evaluated**: a false one lies outside the linear abstraction.

PolySAT - Conflict resolution

With **all** variables assigned and no interval conflict, constraints are **evaluated**: a false one lies outside the linear abstraction.

Example ($k = 4$): $x \cdot y = 1$, with $x \cdot y$ treated as a fresh variable. The search picks $x = 2$, $y = 3$, but $2 \cdot 3 = 6 \neq 1$.

- **Saturation**: $p \cdot q = 1 \Rightarrow \mathrm{parity}(p) = 0$

PolySAT - Conflict resolution

With **all** variables assigned and no interval conflict, constraints are **evaluated**: a false one lies outside the linear abstraction.

Example ($k = 4$): $x \cdot y = 1$, with $x \cdot y$ treated as a fresh variable. The search picks $x = 2$, $y = 3$, but $2 \cdot 3 = 6 \neq 1$.

- **Saturation**: $p \cdot q = 1 \Rightarrow \mathrm{parity}(p) = 0$
- **Incremental linearization**: $p = 0 \Rightarrow p \cdot q = 0$, $p = 2^j \Rightarrow p \cdot q = 2^j q$

PolySAT - Conflict resolution

With **all** variables assigned and no interval conflict, constraints are **evaluated**: a false one lies outside the linear abstraction.

Example ($k = 4$): $x \cdot y = 1$, with $x \cdot y$ treated as a fresh variable. The search picks $x = 2$, $y = 3$, but $2 \cdot 3 = 6 \neq 1$.

- **Saturation**: $p \cdot q = 1 \Rightarrow \mathrm{parity}(p) = 0$
- **Incremental linearization**: $p = 0 \Rightarrow p \cdot q = 0$, $p = 2^j \Rightarrow p \cdot q = 2^j q$
- **Bit-blasting** (last resort)

PolySAT - Conflict resolution

With **all** variables assigned and no interval conflict, constraints are **evaluated**: a false one lies outside the linear abstraction.

Example ($k = 4$): $x \cdot y = 1$, with $x \cdot y$ treated as a fresh variable. The search picks $x = 2$, $y = 3$, but $2 \cdot 3 = 6 \neq 1$.

- **Saturation**: $p \cdot q = 1 \Rightarrow \mathrm{parity}(p) = 0$
- **Incremental linearization**: $p = 0 \Rightarrow p \cdot q = 0$, $p = 2^j \Rightarrow p \cdot q = 2^j q$
- **Bit-blasting** (last resort)

In the example: $\mathrm{parity}(x) = 0$ is then encoded by the constraint $2^{4-1}x \neq 0$, so $x = 2$ is not picked again.

PolySAT - Results

- **Helping Z3**: PolySAT solves 135 problems Z3 does not. Z3 + PolySAT still solves **fewer** than cvc5 over all four benchmark sets on the paper.

PolySAT - Results

- **Helping Z3**: PolySAT solves 135 problems Z3 does not. Z3 + PolySAT still solves **fewer** than cvc5 over all four benchmark sets on the paper.
- **Worse than all other approaches** on SMT-LIB `QF_BV` (46 191 benchmarks, solved within 60 s):

Bit-blasting		**Word-level**	
Bitwuzla	44 948	Yices2-mcsat	36 551
STP	44 473	cvc5-IntBlast	35 627
Yices2	44 189	Z3-IntBlast	35 283
cvc5	42 339	**Z3-PolySAT**	**27 377**
Z3	41 709		

PolySAT - Limitations

- Constraints with &, | and shifts are not polynomials: they produce no forbidden intervals and no propagation.
- They are only checked once **every** variable is assigned.

PolySAT - Limitations

- Constraints with &, | and shifts are not polynomials: they produce no forbidden intervals and no propagation.
- They are only checked once **every** variable is assigned.
- Intervals come only from **linear** constraints; non-linear propagation is skipped (it produced weak lemmas).

Int-blasting - Setting

Convert a bit-vector problem into integer arithmetic [**Zohar et al.**, **2022**]:

- Constants are converted in the expected way ($100_2 \rightarrow 4$).
- Variables become bounded integers (x of width 3: $0 \leq x^{\text{int}} < 2^3$).

Int-blasting - Setting

Convert a bit-vector problem into integer arithmetic [**Zohar et al.**, **2022**]:

- Constants are converted in the expected way ($100_2 \to 4$).
- Variables become bounded integers (x of width 3: $0 \leq x^{\text{int}} < 2^3$).
- Arithmetic operations have convenient translations :) for x, y of width 3, $x + y \to (x^{\text{int}} + y^{\text{int}}) \bmod 2^3$.

Int-blasting - Setting

Convert a bit-vector problem into integer arithmetic [**Zohar et al.**, **2022**]:

- Constants are converted in the expected way ($100_2 \to 4$).
- Variables become bounded integers (x of width 3: $0 \leq x^{\text{int}} < 2^3$).
- Arithmetic operations have convenient translations :) for x, y of width 3, $x + y \to (x^{\text{int}} + y^{\text{int}}) \bmod 2^3$.
- Shifts and structural operators admit similarly simple translations: for x of width 3, $x \ll 2 \to (x^{\text{int}} \cdot 2^2) \bmod 8$.

Int-blasting - Bitwise operations

Key observation: all bitwise operations can be expressed with arithmetic and the bitwise and operation.

Int-blasting - Bitwise operations

Key observation: all bitwise operations can be expressed with arithmetic and the bitwise and operation.

- $x \mid y \rightarrow (x + y) - (x \,\&\, y)$
- $x \oplus y \rightarrow x \mid y - (x \,\&\, y) \rightarrow x + y - 2 \cdot (x \,\&\, y)$
- $\neg x \rightarrow 2^k - 1 - x$ (for width k)

Int-blasting - Bitwise operations

Key observation: all bitwise operations can be expressed with arithmetic and the bitwise and operation.

- $x \mid y \to (x + y) - (x \,\&\, y)$
- $x \oplus y \to x \mid y - (x \,\&\, y) \to x + y - 2 \cdot (x \,\&\, y)$
- $\neg x \to 2^k - 1 - x$ (for width k)

We then only need to encode the bitwise and, written $x \,\&_k\, y$ (**IAND**)!

Int-blasting - **Encoding** `bvand`

Goal: recover each bit from the integer representation using the div and mod operators.

Int-blasting - **Encoding** `bvand`

Goal: recover each bit from the integer representation using the div and mod operators.

$$\mathrm{bsel}_i(x) := (x \,\mathrm{div}\, 2^i) \bmod 2$$

Int-blasting - **Encoding** `bvand`

Goal: recover each bit from the integer representation using the div and mod operators.

$$\mathrm{bsel}_i(x) := (x \,\mathrm{div}\, 2^i) \bmod 2$$

E.g. $x = 6 = 110_2$: $\mathrm{bsel}_1(x) = (6 \,\mathrm{div}\, 2) \bmod 2 = 1$,
$\mathrm{bsel}_0(x) = 6 \bmod 2 = 0$.

Int-blasting - **Encoding** `bvand`

Goal: recover each bit from the integer representation using the div and mod operators.

With this operation, IAND can be encoded in two ways:

- **sum**: rebuild the whole number from the bits
- **bitwise**: one constraint per bit

Int-blasting - **Encoding** `bvand`

Goal: recover each bit from the integer representation using the div and mod operators.

With this operation, IAND can be encoded in two ways:

- **sum**: rebuild the whole number from the bits

$$x \,\&_k\, y = \sum_{i=0}^{k-1} 2^i \cdot \mathrm{ite}\big(\mathrm{bsel}_i(x) = 1 \wedge \mathrm{bsel}_i(y) = 1,\ 1,\ 0\big)$$

- **bitwise**: one constraint per bit

Int-blasting - **Encoding** `bvand`

Goal: recover each bit from the integer representation using the div and mod operators.

With this operation, IAND can be encoded in two ways:

- **sum**: rebuild the whole number from the bits
- **bitwise**: one constraint per bit

$$\bigwedge_{i=0}^{k-1} \mathrm{bsel}_i(x \,\&_k\, y) = \mathrm{ite}\big(\mathrm{bsel}_i(x) = 1 \wedge \mathrm{bsel}_i(y) = 1,\ 1,\ 0\big)$$

Int-blasting - Limitations

- The IAND encoding **grows with the bit-width** and is handled by the **non-linear** arithmetic solver.

Int-blasting - Limitations

- The IAND encoding **grows with the bit-width** and is handled by the **non-linear** arithmetic solver.
- **Bitwise operators have no arithmetic counterpart**: they all go through IAND, i.e., $\mathrm{div}/\mathrm{mod}$ arithmetic.

Int-blasting - Limitations

- The IAND encoding **grows with the bit-width** and is handled by the **non-linear** arithmetic solver.
- **Bitwise operators have no arithmetic counterpart**: they all go through IAND, i.e., $\mathrm{div}/\mathrm{mod}$ arithmetic.
- Relies on non-linear integer arithmetic: weaker on **sat** instances.

Overall limitations

Approach	**Main limitation**
Three-valued abstraction	Loses correlations between operands; no preprocessing.
PolySAT	Bitwise operators and shifts get no forbidden intervals and are only checked once every variable is assigned.
Int-blasting	Recovers the bits with $\mathrm{div}/\mathrm{mod}$ and grows with the bit-width.

Overall limitations

Approach	Main limitation
Three-valued abstraction	Loses correlations between operands; no preprocessing.
PolySAT	Bitwise operators and shifts get no forbidden intervals and are only checked once every variable is assigned.
Int-blasting	Recovers the bits with $\mathrm{div}/\mathrm{mod}$ and grows with the bit-width.

Common thread: each approach handles either arithmetic or bit-level reasoning well, but **not both**.

References i

Biere, A. (**2016**). *Weaknesses of CDCL Solvers*. Invited talk, Theoretical Foundations of SAT Solving Workshop, Fields Institute, Toronto.

Niemetz, A. (**2025**). *Challenges in State-of-the-Art Bit-Precise Reasoning*. Invited talk, Simons Institute and SLMath Joint Workshop: AI for Mathematics and Theoretical Computer Science, Berkeley.

Onderka, J., A. Biere, and M. Fleury (**2026**). "Lean Certified Bitvector Solving without Bitblasting". In: Formal Methods in Computer-Aided Design (FMCAD).

Onderka, J. and S. Ratschan (**2022**). "Fast three-valued abstract bit-vector arithmetic". In: International Conference on Verification, Model Checking, and Abstract Interpretation (VMCAI). Springer, pp. 242–262.

Rath, J. et al. (**2024**). "PolySAT: Word-level Bit-vector Reasoning in Z3". In: Verified Software: Theories, Tools and Experiments (VSTTE). LNCS. Springer.

Zohar, Y. et al. (**2022**). "Bit-Precise Reasoning via Int-Blasting". In: Verification, Model Checking, and Abstract Interpretation (VMCAI). LNCS. Springer.